CropShield AI Design Document

Capstone submission | based on inspected source, configuration and saved reports

Problem statement

Agricultural decisions depend on interacting crop, season, location, soil, weather and input conditions. CropShield AI provides a single interface to search historical crop cases, summarize relevant evidence, recommend crops, estimate yield and surface agronomic risks.

Business objective

Support agronomists, extension officers and agricultural users with faster, evidence-linked decision support. Predictions are advisory estimates and should be reviewed with local agronomic expertise.

Architecture and technology choices

Layer	Implementation	Reason / trade-off
API	Python, FastAPI, Pydantic, Uvicorn	Typed HTTP endpoints and generated OpenAPI docs.
Retrieval	FAISS CPU, local hashed dense features, keyword/metadata search, hybrid ranking	Local operation and reproducibility; hashed vectors are not a pretrained semantic embedding model.
Generation	Optional OpenAI-compatible chat/judge provider	External key, credits, endpoint access, latency and structured output can limit runtime evaluation.
ML	scikit-learn RandomForest classifier and regressor	Fits tabular features; quality reflects synthetic training data and should not be generalized to field outcomes.
UI	FastAPI-served HTML/CSS/JS SPA; separate Streamlit client	Main browser app uses no frontend build step; Streamlit requires its optional dependency.
Storage	CSV, model files, FAISS/index artifacts, SQLite auth, JSON feedback	Simple local persistence; not a distributed production data layer.

Retrieval strategy

The project builds vector and keyword indexes over structured case rows. Retrieval can apply crop, state and season metadata filters, combine dense and lexical evidence, and rerank candidates. Labeled benchmark evaluation compares retrieved crop labels with the query's ground-truth labels using Precision@K, Recall@K, MRR and hit rate.

Agent workflow

The orchestrator applies input/scope checks, classifies the query and extracts conditions, retrieves supporting cases, invokes recommendation, risk, yield and impact components where appropriate, validates output and formats the advisory. The implementation is a modular orchestrated pipeline; component boundaries do not imply independently hosted agents.

Evaluation strategy

Local retrieval and ML reports are persisted under evaluation/reports/. RAGAS and DeepEval adapters exist for per-query LLM judging. Those judge evaluations are provider-dependent and can fail when the configured model endpoint is unreachable or its account has no credits. The report distinguishes saved benchmark evidence from pending live judge results.

Limitations and trade-offs

- The principal 1,500-row dataset is synthetic local test data, not a verified field survey; measured performance is not evidence of real-world agronomic accuracy.

- A saved retrieval benchmark can be biased by its 20 labeled queries and crop-level relevance labels; it does not measure all forms of answer quality.
- LLM judge output is nondeterministic, costs credits and depends on valid endpoint configuration. RAGAS/DeepEval scores are therefore not always available.
- FAISS and local files suit a single-node prototype; access controls, backup, monitoring and data governance need production hardening.
- The Streamlit frontend is an additional client; the primary UI is the static frontend served by FastAPI.